



37-MAVZU · 42 TADAN · IV BOB. PYTHON, POSTGRESQL VA DJANGO

Django Admin panel bilan ishlash. Django Forms yordamida ma'lumotlarni qabul qilish

Superuser, modellarni admin'da ro'yxatdan o'tkazish va sozlash (list_display, search_fields, list_filter, prepopulated_fields); Form va ModelForm, CSRF himoyasi, validatsiya, clean metodlari, xabarlar; ro'yxatdan o'tish va kirish tizimi.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- Django'ning tayyor **admin paneli** orqali ma'lumotlarni boshqarishni va uni sozlashni;
- **Django Forms** — foydalanuvchidan ma'lumotni qabul qilish, tekshirish va bazaga saqlashni;
- **CSRF** hujumi va `{% csrf_token %}` himoyasini;
- O'z validatsiya qoidalarini yozishni;
- **Ro'yxatdan o'tish va kirish** (Login/Register) tizimini (24–25-loyihalar).

Django Admin — tayyor boshqaruv paneli

Har bir saytga "orqa eshik" kerak: maqolalarni qo'shish, tahrirlash, foydalanuvchilarni boshqarish.

Boshqa frameworklarda buni haftalab yozish kerak — Django'da u **tayyor**. Bu Django'ning eng mashhur imkoniyatlaridan biri.

1. Superuser va kirish

Terminal

```
python manage.py createsuperuser
# Username: admin
# Email: admin@maktab.uz
# Password: ***** (yozilayotganda ko'rinmaydi — bu normal)
python manage.py runserver
```

<http://127.0.0.1:8000/admin/> ni oching va kiring. Hozircha faqat "Foydalanuvchilar" va "Guruhlar" bor.

2. Modellarni ro'yxatdan o'tkazish

blog/admin.py

```
from django.contrib import admin
from .models import Kategoriya, Maqola

admin.site.register(Kategoriya)
admin.site.register(Maqola)
```

Sahifani yangilang — "Blog" bo'limida Kategoriyalar va Maqolalar paydo bo'ldi. Endi ularni qo'shish, tahrirlash va o'chirish mumkin — bir qator kod yozmasdan (tayyor formalar, validatsiya, tasdiqlash oynalari)! Ro'yxatda obyektlar `__str__` qaytargan matn bilan ko'rinadi (36-mavzu).

3. Admin'ni sozlash

blog/admin.py

```
from django.contrib import admin
from .models import Kategoriya, Maqola

@admin.register(Kategoriya)
class KategoriyaAdmin(admin.ModelAdmin):
    list_display = ["nomi", "slug"]
    prepopulated_fields = {"slug": ("nomi",)} # nom yozilganda slug
    avtomatik to'ladi

@admin.register(Maqola)
class MaqolaAdmin(admin.ModelAdmin):
    list_display = ["sarlavha", "kategoriya", "muallif", "chop_etilgan",
                    "korishlar", "yaratilgan"]
    list_filter = ["chop_etilgan", "kategoriya", "yaratilgan"] # o'ng
    tomonda filtrlar
    search_fields = ["sarlavha", "matn"] # qidiruv
    maydoni
    list_editable = ["chop_etilgan"] #
    ro'yxatning o'zida tahrirlash
    list_per_page = 20
    date_hierarchy = "yaratilgan" # sana
    bo'yicha navigatsiya
    readonly_fields = ["korishlar", "yaratilgan", "yangilangan"]
    actions = ["chop_etish"]

@admin.action(description="Tanlanganlarni chop etish")
def chop_etish(self, request, queryset):
    soni = queryset.update(chop_etilgan=True)
    self.message_user(request, f"{soni} ta maqola chop etildi")

admin.site.site_header = "Maktab blogi – boshqaruv paneli"
admin.site.site_title = "Maktab blogi"
```

Sozlama

Vazifasi

<code>list_display</code>	Ro'yxatda qaysi ustunlar ko'rinadi
<code>list_filter</code>	O'ng paneldagi filtrlar
<code>search_fields</code>	Qidiruv qaysi maydonlar bo'yicha
<code>list_editable</code>	Ro'yxatning o'zida o'zgartiriladigan maydonlar
<code>prepopulated_fields</code>	Bir maydondan boshqasini avtomatik to'ldirish (slug)
<code>readonly_fields</code>	Faqat o'qish uchun
<code>actions</code>	Tanlangan obyektlar ustida ommaviy amallar



Foydalanuvchilar va ruxsatlar

Admin'da yangi foydalanuvchi yaratib, unga **"Staff status"** bering va faqat "Maqola" uchun "qo'shish/o'zgartirish" ruxsatlarini belgilang — u admin'ga kira oladi, lekin kategoriyalar va foydalanuvchilarni ko'rmaydi. Masalan, sinf muxbirlari uchun.

Django Forms — nima uchun?

Admin panel — faqat ma'murlar uchun. Oddiy foydalanuvchilar (izoh qoldirish, ro'yxatdan o'tish, maqola yuborish) uchun **sayt ichida** formalar kerak. 3-mavzuda HTML formalarni yozdik va 20-mavzuda JavaScript bilan tekshirdik. Lekin brauzerdagi tekshiruvni chetlab o'tish oson — **server ham tekshirishi shart**.

Django Forms uchta ishni bajaradi:

1. **HTML yaratadi** — input'lar, label'lar avtomatik;
2. **Tekshiradi (validatsiya)** — majburiy maydonlar, email formati, uzunlik, o'z qoidalarinigiz;
3. **Tozalaydi** — matnni kerakli turga (son, sana) aylantiradi va xavfsiz qiladi.

Oddiy Form: "Aloqa" formasi

blog/forms.py (yangi fayl)

```
from django import forms

class AloqaForm(forms.Form):
    ism = forms.CharField(label="Ismingiz", max_length=100)
    email = forms.EmailField(label="Email")
    mavzu = forms.ChoiceField(label="Mavzu", choices=[
        ("taklif", "Taklif"),
        ("xato", "Xato haqida xabar"),
        ("boshqa", "Boshqa"),
    ])
    xabar = forms.CharField(label="Xabar", widget=forms.Textarea(attrs={"rows":
5}), min_length=10)

    def clean_ism(self):                                # bitta maydon uchun o'z
tekshiruvimiz
        ism = self.cleaned_data["ism"].strip()
        if any(belgi.isdigit() for belgi in ism):
            raise forms.ValidationError("Ismda raqam bo'lmashligi kerak")
        return ism.title()
```

```

from django.contrib import messages
from django.shortcuts import render, redirect
from .forms import AloqaForm

def aloqa(request):
    if request.method == "POST":
        form = AloqaForm(request.POST)
        to'ldiramiz
        if form.is_valid():
            o'tdi
            data = form.cleaned_data
            (dict)
            print("Yangi xabar:", data["ism"], data["email"], data["xabar"])
            messages.success(request, "Rahmat! Xabaringiz yuborildi.")
            return redirect("blog:aloqa")
            uchun redirect
            # ✗ xato bo'lsa – forma xatolar bilan qayta ko'rsatiladi
        else:
            marta ochildi
            form = AloqaForm()
            return render(request, "blog/aloqa.html", {"form": form})
            # bo'sh forma
            # forma yuborildi
            # kelgan ma'lumot bilan
            # ✔ hamma tekshiruvdan
            # tozalangan ma'lumot
            # qayta yuborilmasligi
            # GET – sahifa birinchi

```

blog/templates/blog/aloqa.html

```
{% extends "blog/base.html" %}

{% block content %}
    <h1>Biz bilan bog'laning</h1>

    {% if messages %}
        {% for msg in messages %}<p class="xabar">{{ msg }}</p>{% endfor %}
    {% endif %}

    <form method="post" novalidate>
        {% csrf_token %}                                {# ← MAJBURIY: CSRF himoyasi #}
        {{ form.as_p }}                                  {# barcha maydonlar <p> ichida, xatolar
bilan #}
        <button type="submit">Yuborish</button>
    </form>
{% endblock %}
```

blog/urls.py

```
path("aloqa/", views.aloqa, name="aloqa"),
```



Django formasini qayta ishlashning standart sxemasi



CSRF nima?

CSRF (Cross-Site Request Forgery) — yovuz sayt sizning brauzeringiz nomidan boshqa saytga yashirin forma yuboradigan hujum (masalan, siz tizimga kirgan blogga "barcha maqolalarni o'chir" so'rovi). `{% csrf_token %}` formaga yashirin maxfiy kod qo'shadi; Django POST so'rovni faqat shu kod to'g'ri bo'lsa qabul qiladi. Usiz: **403 Forbidden — CSRF verification failed.**

Formani qo'lda chiqarish (dizayn uchun)

Django shablon

```
{% for field in form %}
  <div class="maydon">
    {{ field.label_tag }}
    {{ field }}
    {% if field.errors %}<small class="xato">{{ field.errors.0 }}</small>{%
endif %}
  </div>
{% endfor %}
```

ModelForm — modeldan avtomatik forma

Agar forma **model**ga ma'lumot saqlasa, maydonlarni qayta yozish shart emas — `ModelForm` ularni modeldan o'zi oladi.

```
from .models import Maqola

class MaqolaForm(forms.ModelForm):
    class Meta:
        model = Maqola
        fields = ["sarlavha", "kategoriya", "matn"] # foydalanuvchi
        to'ldiradigan maydonlar
        widgets = {
            "sarlavha": forms.TextInput(attrs={"placeholder": "Qiziqarli
sarlavha..."}),
            "matn": forms.Textarea(attrs={"rows": 8}),
        }

    def clean_sarlavha(self):
        s = self.cleaned_data["sarlavha"]
        if len(s) < 5:
            raise forms.ValidationError("Sarlavha kamida 5 belgidan iborat
bo'lsin")
        return s
```

```

from django.contrib.auth.decorators import login_required
from .forms import MaqolaForm

@login_required                                     # faqat tizimga kirganlar
uchun
def maqola_yozish(request):
    if request.method == "POST":
        form = MaqolaForm(request.POST)
        if form.is_valid():
            maqola = form.save(commit=False)        # hali bazaga yozmay
            maqola.muallif = request.user           # muallifni o'zimiz
            maqola.chop_etilgan = False             # admin tasdiqlaguncha
            maqola.save()                           # endi saqlaymiz
            messages.success(request, "Maqola yuborildi! Admin tasdiqlagach
            chop etiladi.")
            return redirect("blog:bosh")
        else:
            form = MaqolaForm()
            return render(request, "blog/maqola_form.html", {"form": form})

```

`commit=False` — formadan obyekt yaratadi, lekin bazaga saqlamaydi: shunda formada bo'lmagan maydonlarni (`muallif`) qo'shib, keyin saqlash mumkin.

Ro'yxatdan o'tish va kirish (Login / Register)

Django'da foydalanuvchilar tizimi tayyor: `User` modeli, parollarni **xeshlab** (shifrlab bo'lmaydigan ko'rinishda) saqlash, kirish/chiqish view'lari.

Kirish va chiqish — tayyor view'lar

config/urls.py

```
urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/", include("django.contrib.auth.urls")), # login/,
    path("logout/", password_change/ ...
    path("", include("blog.urls")),
]
```

config/settings.py

```
LOGIN_REDIRECT_URL = "blog:bosh" # kirgandan keyin
LOGOUT_REDIRECT_URL = "blog:bosh" # chiqqandan keyin
LOGIN_URL = "login" # @login_required kirmaganlarni shu
yerga yuboradi
```

templates/registration/login.html (loyiha darajasida yoki blog/templates/registration/)

```
{% extends "blog/base.html" %}
{% block content %}
    <h1>Kirish</h1>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button>Kirish</button>
    </form>
    <p>Akkauntingiz yo'qmi? <a href="{% url 'blog:royxat' %}">Ro'yxatdan
o'ting</a></p>
{% endblock %}
```

Ro'yxatdan o'tish

blog/views.py

```
from django.contrib.auth import login
from django.contrib.auth.forms import UserCreationForm

def royxatdan_otish(request):
    if request.method == "POST":
        form = UserCreationForm(request.POST)      # tayyor forma: username +
        parol + tasdiqlash
        if form.is_valid():
            user = form.save()                      # parol avtomatik xeshlanadi
            login(request, user)                   # darhol tizimga kiritamiz
            messages.success(request, f"Xush kelibsiz, {user.username}!")
            return redirect("blog:bosh")
        else:
            form = UserCreationForm()
    return render(request, "blog/royxat.html", {"form": form})
```

blog/urls.py (qo'shimcha)

```
path("royxat/", views.royxatdan_otish, name="royxat"),
path("yozish/", views.maqola_yozish, name="yozish"),
```

base.html menyusiga

```
{% if user.is_authenticated %}
<span>👤 {{ user.username }}</span>
<a href="{% url 'blog:yozish' %}">✍️ Maqola yozish</a>
<form method="post" action="{% url 'logout' %}" style="display:inline">
    {% csrf_token %}<button>Chiqish</button>           {# Django 5'da logout
- POST so'rov #}
</form>
{% else %}
<a href="{% url 'login' %}">Kirish</a>
<a href="{% url 'blog:royxat' %}">Ro'yxatdan o'tish</a>
{% endif %}
```



Parollar bazada **ochiq holda saqlanmaydi** — pgAdmin'da `auth_user` jadvalining `password` ustuniga qarang: `pbkdf2_sha256$...` kabi xesh. Hatto admin ham parolingizni bilmaydi. Hech qachon parolni o'zingiz oddiy `CharField` da saqlamang!

Amaliy mashg'ulot

- 1 Admin'ni sozlang (yuqoridagi `KategoriyaAdmin`, `MaqolaAdmin`) va admin orqali 5 ta maqola qo'shing. Filtr, qidiruv va ommaviy amalni sinang.
- 2 "Aloqa" formasini yarating. Bo'sh, noto'g'ri email va raqamli ism bilan yuborib, xato xabarlarini ko'ring. `{% csrf_token %}` ni olib tashlab, 403 xatoni ko'ring — keyin qaytaring.
- 3 Aloqa xabarlarini bazada saqlang: `Xabar` modeli (ism, email, mavzu, matn, sana) → `ModelForm` → admin'da ko'rinsin.
- 4 Ro'yxatdan o'tish (24-loyiha) va kirish/chiqish (25-loyiha) tizimini qo'shing; menyuda foydalanuvchi holatiga qarab havolalar o'zgarsin.
- 5 "Maqola yozish" sahifasini (`@login_required`) yarating. Tizimga kirmasdan `/yozish/` ga kirib ko'ring — login sahifasiga yo'naltiradi.

Ko'p uchraydigan xatolar

- ✗ `CSRF verification failed` — formada `{% csrf_token %}` yo'q yoki `method="post"` yozilmagan.
- ✗ Forma yuborilgach sahifa yangilansa, ma'lumot **qayta yuboriladi** — muvaffaqiyatli POST'dan keyin doim `redirect`.
- ✗ `form.cleaned_data` ni `is_valid()` dan **oldin** ishlatish — `AttributeError`.
- ✗ Admin'da model ko'rinmayapti — `admin.py` da ro'yxatdan o'tkazilmagan yoki server qayta ishga tushirilmagan.
- ✗ ModelForm'da `fields` ko'rsatilmagan — Django xato beradi (xavfsizlik uchun barcha maydonlarni avtomatik ochmaydi).

O'zingizni tekshiring

? Modelni admin panelda ko'rsatish uchun nima qilinadi?

- A. `settings.py` ga qo'shiladi
- B. `admin.py` da `admin.site.register(Model)`
- C. Migratsiya qilinadi
- D. Shablon yaratiladi

? Admin ro'yxatida qaysi ustunlar ko'rinishini belgilovchi sozlama?

- A. `fields`
- B. `list_filter`
- C. `search_fields`
- D. `list_display`

? POST formada majburiy bo'lgan xavfsizlik tegi?

- A. `{% csrf_token %}`
- B. `{% secure %}`
- C. `{% load form %}`
- D. `{% token %}`

? **Forma ma'lumotlari to'g'riligini tekshiruvchi metod?**

- A. `form.save()`
- B. `form.check()`
- C. `form.is_valid()`
- D. `form.validate()`

? **Model asosida avtomatik forma yaratuvchi klass?**

- A. `forms.Form`
- B. `forms.ModelForm`
- C. `admin.ModelAdmin`
- D. `models.Form`

? **View'ni faqat tizimga kirgan foydalanuvchilar uchun ochiq qiluvchi dekorator?**

- A. `@admin_only`
- B. `@csrf_exempt`
- C. `@staff`
- D. `@login_required`



Uyga vazifa

1. **Izohlar:** `Izoh` modeli (maqola FK, muallif FK, matn, sana). Maqola sahifasida izohlar ro'yxati va faqat tizimga kirganlar uchun izoh formasi.
2. Kutubxona loyihangiz (22-loyiha) uchun admin'ni to'liq sozlang: kitoblar ro'yxatida muallif, janr, yil; filtr — janr va mavjudlik; qidiruv — nom va muallif.

✓ Yodda tuting

- Admin: `createsuperuser`; `@admin.register(Model)` + `ModelAdmin` (`list_display`, `list_filter`, `search_fields`...).
- Forma sxemasi: GET → bo'sh forma; POST → `Form(request.POST)` → `is_valid()` → `cleaned_data` / `save()` → `redirect`.
- `{% csrf_token %}` — har bir POST formada; `clean_maydon()` — o'z tekshiruvlaringiz.
- `ModelForm` + `save(commit=False)`; `messages` — foydalanuvchiga xabar.
- Auth: `include("django.contrib.auth.urls")`, `UserCreationForm`, `login()`, `@login_required`, `user.is_authenticated`.

□ Darsni o'qib bo'ldim

✓ Test javoblari

1. Modelni admin panelda ko'rsatish uchun nima qilinadi? — **B) admin.py da admin.site.register(Model)**
2. Admin ro'yxatida qaysi ustunlar ko'rinishini belgilovchi sozlama? — **D) list_display**
3. POST formada majburiy bo'lgan xavfsizlik tegi? — **A) {% csrf_token %}**
4. Forma ma'lumotlari to'g'riligini tekshiruvchi metod? — **C) form.is_valid()**
5. Model asosida avtomatik forma yaratuvchi klass? — **B) forms.ModelForm**
6. View'ni faqat tizimga kirgan foydalanuvchilar uchun ochiq qiluvchi dekorator? — **D) @login_required**